

Suite E2E — universo completo sobre la demo Merlo (QA)

Corrida final `E2E_RUN_ID=final` · 2026-08-23 · ambiente real `munify-qa.netlify.app` + Cloud Run `munify-api-qa`
+ DB `sugerenciasmun-qa` · fuente: `frontend/e2e/.full-run-merlo.log` y verificaciones directas contra la DB de QA

1 Resumen ejecutivo

CORRIDA FINAL

59 / 59

0 fallos · 23,1 min ·
medido

ACTORES
EJERCITADOS

4

vecino · supervisor ·
operario · admin

HALLAZGOS DE LA
APP

6

2 serios de plata ·
detalle en \$5

COSTO POR RE-
CORRIDA

0 tokens

un comando · ~23
min de máquina

El circuito entero del municipio quedó probado de punta a punta, con browser real, contra QA desplegado. La suite es determinística (matriz fija, sin IA, sin randomness), agnóstica de tenant (otro municipio = otro JSON) y de costo agéntico cero en cada corrida futura. El método quedó portable en `base-compartida/19-SUITE-E2E-AGNOSTICA.md`.

2

Hito cero — la empresa nueva

La campaña arrancó creando una empresa desde el generador real de demos (`POST /municipios/crear-demo`), como pidió el dueño: la creación misma es parte de lo probado.

QUÉ	VALOR	VERIFICACIÓN
Municipio	Merlo (Buenos Aires) · muni <code>153</code> · código <code>merlo</code>	API pública QA
Acceso	<code>munify-qa.netlify.app/merlo</code> — botonera pública, PIN 1680 al tocar un perfil	browser real 9/9
Protección	El PIN es la password real de los 13 usuarios demo; <code>demo123</code> rebota con 401; el DELETE de la demo también exige PIN	curl login OK/401
Locaciones	60 direcciones reales de Merlo (Nominatim reverse; ej. "Belén 2281", "Avenida Domingo Sica")	DB QA
Semilla	13 reclamos · 14 solicitudes · 10 OT · 15 ítems de inventario · 24 turnos · 12 dependencias · 13 trámites (cobro al inicio Y al final) · tesorería	conteo en DB QA

3

La matriz — 59 pruebas, 5 capas

CAPA	CASOS	QUÉ PRUEBA	RESULTADO
Login con PIN (setup)	9	Un login por rol vía el picker: admin, 5 supervisores, 2 vecinos, operario — cada uno atraviesa el modal de PIN	9/9
Campo (4 actores)	8	Alta de recursos → OT con reserva de camión + tornillos planeados, asignada al operario → exclusividad del activo → el operario completa con consumo REAL (100→80, no descuenta lo planeado) → el SUPERVISOR finaliza el reclamo (la cuadrilla no cierra) → cierre PARCIAL (OT con 2 reclamos, uno queda abierto) → cancelar libera y NO descuenta → negativo de permisos	8/8
Cobros	2	Cobro al inicio: la solicitud nace pendiente de pago, el vecino paga por el checkout y el cobro llega a la pantalla Cobros del admin. Cobro al final: entra sin pedir plata en la puerta	2/2
Reclamos	20	Wizard del vecino (categoría manual, dirección real) → gestor (admin en impares, supervisor del área en pares) → variantes: asignación simple, orden de trabajo con cascada, huérfano → el vecino ve el cierre	20/20
Trámites + pago	20	Solicitud → paga si cobra al inicio (checkout mock) → turno para ESA solicitud → verificación de documentos obligatorios → paga al final si corresponde (el backend bloquea con 400 el cierre impago) → finalizar → el vecino ve el cierre	20/20

59 de 59 en 23,1 minutos (log de la corrida, medido). Un solo worker, sin retries: el orden es parte de lo probado.

4

Cómo se llegó al verde — veedor + fixer

Corrida m1: 39 verdes, trámites 0/6 →

Veedor corta la corrida (4 min/fallo condenado) y triangula contra la DB →

Fixer (Opus) diagnostica y arregla SOLO tests →

Trámites 20/20 →

Veedor verifica con corrida propia →

Final 59/59

CAUSA RAÍZ (BUGS DEL TEST, NO DE LA APP)

FIX

El botón "Verificado sin archivo" DESAPARECE al verificar: iterar por índice pedía un elemento inexistente y el click sin timeout colgaba el caso 240 s

Se clickea siempre el primero; la señal real es que la cuenta baje

El checkout tiene 5 formas según el medio de pago; el helper solo conocía el selector — en adhesión a débito (medio único, sin selector) esperaba un botón que no existe

`caminarCheckoutMock()` cubre las 5 formas (CBU + TyC incluidos)

El cobro AL FINAL nunca se había ejercitado: el backend rechaza con 400 el cierre de un trámite impago

Paso de pago del vecino entre "en curso" y "finalizar"

Fe de erratas del veedor: en el análisis previo se reportó que "nada impide finalizar una solicitud impaga" (G10). Era INCORRECTO: el gate existe en el backend (`api/tramites.py:1151`) y de hecho fue lo que hizo fallar los casos de cobro al final. Lo que sí falta es del lado visual (§5, hallazgo 4).

5

Hallazgos de la APP (sin tocar — decisión del dueño)

Los dejó el circuito real; ninguno sale con tests de endpoints. Orden sugerido de ataque: 1 → 2 → 4 → 3 → 6 → 5.

#	HALLAZGO	DÓNDE	SEVERIDAD
1	Override de testing vivo en PRODUCCIÓN: todo cupón de WhatsApp del Mostrador cobra \$1 en lugar del costo real, en todos los municipios. Verificado en <code>origin/master</code> (no es hipótesis)	<code>backend/api/pagos.py:299</code>	plata / prod
2	Doble cobro posible: <code>crear_sesion_tramite</code> no valida sesión ya aprobada, y el front pinta "Pagar trámite" mientras la request de estado está en vuelo (y un catch silencioso lo deja para siempre si falla). Evidencia: solicitud con "Intentos: 2" en la corrida	<code>backend/api/pagos.py ~222</code> <code>MisTramites.tsx:946</code>	plata (con MP real)
3	Acción fantasma: al verificar el último documento la solicitud avanza sola a "en curso", pero el sheet no se refresca: el operador aprieta "Poner en Curso" sobre algo ya en curso y el front le canta éxito igual	<code>GestionTramites.tsx:1944</code>	UX gestor
4	"Finalizar" habilitado sobre un trámite que el propio panel dice impago: el backend contesta 400 y el operador se come un toast de error — el botón debería estar deshabilitado	<code>GestionTramites.tsx:2427</code>	UX gestor
5	El <code>Sheet</code> (side panel de toda la app) no cierra con Escape	<code>components/ui/Sheet.tsx</code>	UX global
6	Semántica rara en la demo: "Renovación de licencia" configurada como débito automático <i>mensual</i> — para un trámite de pago único, en una demo comercial se lee mal	<code>services/seed_demo.py</code>	demo

6 Lo construido en la jornada (commits en `qa`)

COMMIT	QUÉ
<code>20486a9</code>	Demos protegidas por PIN (el PIN es la password real; modal en la botonera; DELETE con PIN)
<code>fa885b2</code>	Fixes que destraban Mercado Pago REAL: el webhook resuelve la sesión por <code>external_reference</code> , back_urls absolutas, camino único de aprobación (deuda Y solicitud) y gate del endpoint público <code>/confirmar</code>
<code>6342877</code>	Cache de 60 locaciones reales de Merlo + login E2E con PIN
<code>329ac85</code>	Tenant Merlo + <code>campo.spec</code> (4 actores) + <code>pagos.spec</code>
<code>86de043</code>	Fix de trámites del fixer (verificado independientemente): 20/20
<code>e2db58c</code>	Docs de la suite + método portable en <code>base-compartida/19-SUITE-E2E-AGNOSTICA.md</code>

Además, en el ambiente QA: migraciones de proveedores de pago + webhooks + `demo_protegido` ejecutadas en `sugerenciasmun-qa` , y `WEBHOOK_BASE_URL` + `FERNET_KEY` configuradas en el servicio Cloud Run `munify-api-qa` .

7 Próximas capas

CAPA	QUÉ FALTA	QUIÉN
Smoke MP real	El dueño carga sus credenciales en <code>/gestion/proveedores-pago</code> de Merlo (URL directa, PIN 1680) y se prueba un pago real con el webhook firmado — los fixes ya están deployados en QA	<code>dueño + veedor</code>
Decisiones sobre \$5	Priorizar los 6 hallazgos (los dos rojos tocan plata)	<code>dueño</code>
Tasas del padrón	La semilla ya genera partidas y deudas; nadie las ejerce end-to-end aún	<code>suite</code>
Modo veedor / cupón agnóstico	Hoy no existe en la app (solo el caso CENAT hardcodeado): decidir si se generaliza y después se testea	<code>producto</code>

